

GREClue: Failure Indexing with Graph-Based Failure Representation and Entropy-Based Deep Clustering

ZHENYU YANG, Shandong University, China

ZHONGXING YU*, Shandong University, China

Failure indexing aims to group multiple failures according to their root causes and is an essential step in parallel debugging. Failure indexing consists mainly of two steps: failure representation and failure clustering. While many research efforts have been devoted to these two steps, serious issues still exist. For failure representation, existing works use coverage or program memory information, which unfortunately can not capture deep failure semantic. For failure clustering, advanced failure indexing methods use clustering algorithms with preset cluster centers, but this kind of clustering algorithm can handle spherical clusters well but performs poorly when handling clusters of other shapes. To address these issues, this paper proposes GREClue, a novel failure indexing approach with **Graph-based failure Representation and Entropy-based deep Clustering**. GREClue overcomes the issues in order. For failure representation, GREClue designs the failure semantic graph (FSG), a new graph representation that effectively contains semantic information and runtime information of failures. Based on FSG, GREClue further consists of an entropy-based deep clustering component, which can accurately cluster failed tests without presetting cluster centers. An extensive evaluation of GREClue shows that compared to the state-of-the-art failure indexing method, GREClue improves both the performance of estimating the number of faults and the clustering effectiveness by 10% to 41%. Moreover, it has also been shown that GREClue can effectively facilitate parallel debugging.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**.

Additional Key Words and Phrases: Failure indexing, Parallel debugging, Fault localization, Code representation, Entropy, Deep clustering

ACM Reference Format:

Zhenyu Yang and Zhongxing Yu. 2026. GREClue: Failure Indexing with Graph-Based Failure Representation and Entropy-Based Deep Clustering. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE111 (July 2026), 24 pages. <https://doi.org/10.1145/3808118>

1 Introduction

Debugging is an expensive yet frustrating process in software development [60, 64], and fault localization in particular is among the most tedious and time-consuming debugging activities [73]. Worse still, real-life software is unfortunately plagued with multiple bugs, which can deteriorate the performance of fault localization techniques as they typically target the single fault scenario [15, 21]. To address the issue, researchers have investigated the topic of failure indexing [16, 25, 45, 53–56], whose aim is to group failures (*i.e.*, failing tests) according to their root causes. An ideal failure indexing technique can make the number of groups generated equal to the number of faults (*i.e.*, correct estimation of fault number) and the failed tests in the same group originate from the same fault (*i.e.*, high clustering effectiveness). With such failure indexing techniques, developers can

*Zhongxing Yu is the corresponding author.

Authors' Contact Information: Zhenyu Yang, Shandong University, Qingdao, China, yangzyys@mail.sdu.edu.cn; Zhongxing Yu, Shandong University, Qingdao, China, zhongxing.yu@sdu.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2994-970X/2026/7-ARTFSE111

<https://doi.org/10.1145/3808118>

effectively perform parallel debugging [30]. That is, each fault-focused group can be assigned to a certain developer and thus multiple bugs can be localized independently and simultaneously.

The effectiveness of failure indexing techniques predominantly determines the feasibility and performance of parallel debugging, both over- and under-division can result in wasted debugging efforts. Failure indexing consists mainly of two steps [36]: failure representation and failure clustering. Failure representation aims at extracting the signatures of failures, which are then viewed as the failure proxies. Failure clustering employs clustering algorithms to group failures according to the culprit root causes, and typically incorporates distance information about the similarity between failure proxies. Usually, the characteristics of the failure proxies determine the distance metric used. Thus, the core of failure indexing lies in defining a failure representation that is representative enough and then designing a proper clustering algorithm to fit the proxy characteristics.

Accordingly, many research efforts have been devoted to failure representation and failure clustering in recent years [16, 25, 45, 53]. For failure representation, two popular ways are code coverage (CC)-based representation and statistical debugging (SD)-based representation. The CC-based method uses coverage information, in terms of both binary coverage and execution frequency, as failure proxy, and Euclid distance is typically used as the distance metric. On top of fault localization techniques (e.g., spectrum-based ones), the SD-based method employs coverage information to further generate a ranked list of program entities and uses the ranked list as failure proxy. For SD-based methods, the Kendall tau distance and Jaccard distance are typically used as the distance metrics. More recently, in light of the drawbacks of using coverage information to represent failures, Song et al. [54, 55] propose using program memory information (including majorly variable values of highly suspicious statements) to represent failures. Correspondingly, they design a customized distance metric. In terms of failure clustering algorithms, the most advanced failure indexing methods mainly cluster by presetting cluster centers and calculating the distance between data points and cluster centers. For example, Gao et al. [22] propose a new K -medoids algorithm that improves performance by computing latent scores for data points to estimate the most probable combination of cluster medoids. Compared to the original K -medoids algorithm, Song et al. [54, 55] use a mountain function to set cluster centers.

While these research efforts have achieved exciting results, state-of-the-art failure indexing techniques still have some drawbacks. First, current failure representation methods do not capture deep failure semantic and the obtained failure proxies are thus not representative enough. Unlike natural languages, programming languages have formal structures that contain important semantic information. Unfortunately, current failure representation methods poorly exploit this property. The CC-based and SD-based methods essentially rely on coverage information, but being covered is a necessary but not sufficient condition for failure triggering according to the PIE model [65]. Thus, coverage cannot provide a deep insight about failure signature. Program memory information partially encodes program state, and thus offers deeper insight about failure signature than coverage information. However, variable value is a coarse-grained and incomplete characterization of program state. Second, advanced failure indexing methods use clustering algorithms with preset cluster centers (e.g., K -medoids [22]), but works [1, 24, 70] have shown that this kind of clustering algorithm can handle spherical clusters well but performs poorly when handling clusters of other shapes. The data clusters in the failure indexing task have various shapes, leading to poor performance of clustering algorithms based on cluster centers. Thus, the failure indexing task requires an advanced clustering method to adapt to the various shapes of the failure data clusters.

To overcome these drawbacks, we propose GREClue, a novel failure indexing approach with **G**raph-based failure **R**epresentation and **E**ntropy-based deep **C**lustering. GREClue overcomes the drawbacks in order. For failure representation, GREClue features the design of failure semantic graph (FSG), a new graph representation that effectively contains semantic information and runtime

information of failures. In FSG, nodes can represent code methods, code lines, variables, and failed tests, and edges can represent call flow, data flow, membership, and coverage relationship. We aim to leverage the rich information in the FSG to offer deeper insight about failure signature than current methods, thereby enhancing the distinctions between failure features caused by different underlying faults. Based on the FSG, GREClue further consists of an entropy-based deep clustering component, which can accurately cluster failed tests with unknown number of faults without presetting cluster centers. Specifically, the clustering procedure is divided into two processes: cluster merge and cluster split. We use entropy to measure the clustering results in both intra-cluster and inter-cluster aspects. When the entropy value in a cluster is too large, it means that the diversity within the cluster is large and the cluster needs to be split into subclusters. When the entropy value between clusters is too small, it means that the two clusters are similar and the clusters need to be merged. The clustering process stops until the clusters are no longer merged or split. The final number of clusters is the predicted number of faults.

To verify the effectiveness of GREClue, we conducted experiments using two widely used datasets in the field, including Defects4J-Multifault [6, 55] and SIR datasets [19]. The results show that compared to the state-of-the-art failure indexing method SURE [54], GREClue improves both the performance of estimating the number of faults and the clustering effectiveness by 10% to 41%. We also applied the results of GREClue to the parallel debugging scenario, and the results suggest that GREClue can effectively assist in parallel debugging.

In summary, our primary contributions are as follows.

- We propose a novel graph-based failure representation called Failure Semantic Graph (FSG), which captures semantic information and runtime information of failures in a compact way.
- We propose a new entropy-based deep clustering procedure that can complete failure clustering through cluster merging and cluster splitting processes without knowing the number of faults and presetting cluster centers.
- We propose GREClue, a novel failure indexing approach based on the FSG and entropy-based deep clustering procedure. The extensive evaluation shows that GREClue achieves state-of-the-art performance in failure indexing tasks and effectively facilitates parallel debugging.

2 Background

2.1 Parallel Debugging

As the scale and complexity of software systems constantly increase, there in general can be multiple culprit faults when a certain program fails on a test suite. To reduce the overall time to a failure-free program, a feasible way for this scenario is letting multiple developers simultaneously debug the program, *i.e.*, creating parallel work flows. Note that there typically will be more than one developer available to debug a program in practice, particularly in pressing cases such as the release deadline. This technique of paralleling debugging efforts is called *parallel debugging*. As discussed in Section 1, the effectiveness of failure indexing techniques predominantly determines the feasibility and performance of parallel debugging. We next give more details about the two key steps of failure indexing: failure representation and failure clustering.

2.2 Failure Representation

As the directly available information of a failed test (*i.e.*, test input and test output label), is typically too abstract to be directly used for clustering, it is a prerequisite for translating failed tests into a structured and formal form. During the test run, diverse run-time information will be generated, and numerous failure representation methods have been proposed on top of the run-time information. The three most commonly used methods are as follows.

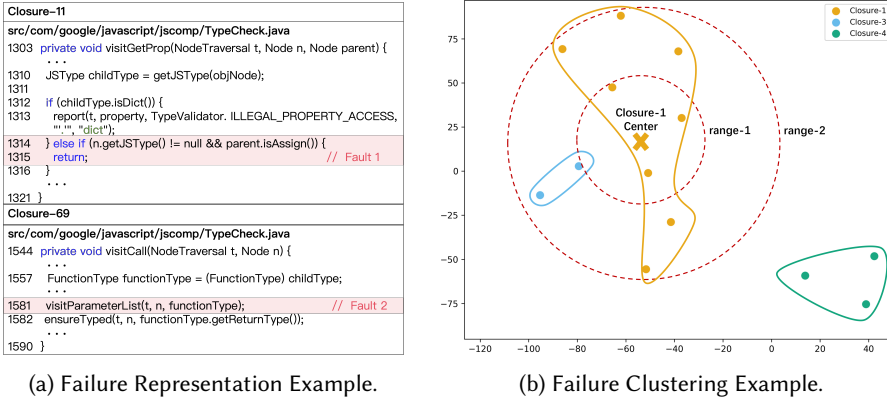


Fig. 1. Motivating Examples for Failure Representation and Failure Clustering.

- **CC-based failure representation.** The CC-based failure representation uses a numerical vector of code coverage to represent a failure. In particular, the length of the created vector is equal to the number of executable program entities (e.g., statements). The value of the i -th element is set as 1 if the execution of a failed test has covered the i -th program entity, and 0 otherwise [17, 28].
- **SD-based failure representation.** The SD-based failure representation uses a ranked list of suspicious program entities to represent a failure. More specifically, for each failed test, the method combines it with all successful tests and then uses a certain fault localization technique to produce a ranked list of potentially faulty program entities [20, 22, 30, 72, 74].
- **Program memory based failure representation.** The program memory based failure representation uses runtime memory information (including majorly variable values) collected during the execution of a failed test to represent the failure [54, 55]. More specifically, SBFL (spectrum-based fault localization) techniques are firstly used to identify some highly-suspicious program statements, which are then set as breakpoints. After that, the runtime memory information at the preset breakpoints is collected during the execution of a failed test.

While these failure representation methods are promising, they do not capture the deep failure semantic well and cannot provide a deep insight about failure signature, as mentioned in Section 1.

2.3 Failure Clustering Algorithm

The characteristics of the failure proxies produced by failure representation methods determine the distance metrics used to measure the distance among the proxies (e.g., Euclid distance, Kendall tau distance, and Jaccard distance), and the failure clustering algorithms aim to group failures with the same triggering cause into the same group based on the distance information. Currently, the most advanced failure clustering algorithms mainly cluster by presetting cluster centers and calculating the distance between the data points and cluster centers [22, 55]. However, as mentioned in Section 1, these clustering algorithms are difficult to adapt to diverse failure clustering scenarios.

3 Illustrating Examples

In this section, we use two real-world examples to explain in detail the limitations of the existing failure representation methods and failure clustering methods.

Limitations of Failure Representation Methods. We illustrate the limitations of CC-, SD-, and program memory-based failure representation methods (as described in Section 1) through a real-world case Closure-11-69, which is shown in Figure 1(a). Closure-11-69 contains two faults (Closure-11 and Closure-69, denoted as f_1 and f_2 respectively) and five failing tests (denoted as

FT_1 to FT_5). Here, FT_1 and FT_2 are triggered by f_1 , while FT_3 , FT_4 , and FT_5 are triggered by f_2 . Ideally, failure indexing should group these failing tests into two distinct clusters $\{FT_1, FT_2\}$ and $\{FT_3, FT_4, FT_5\}$. While FT_1/FT_2 and $FT_3/FT_4/FT_5$ are triggered by different faults, they can share highly similar coverage profiles. For example, the covered code for FT_2 and FT_4 overlaps by 95%. Note that this situation is unfortunately very common in practice as demonstrated by the prestigious work [35] in the domain. Both the CC-based and SD-based methods essentially rely on coverage information as failure proxies, thus they can not differentiate the 5 failing tests well. Program memory-based methods distinguish failures by extracting values of involved variables for highly suspicious statements. However, the highly similar coverage profile typically leads to similar highly suspicious statements and further similar involved variables across different failing tests. For example, the top 10% statements of suspiciousness lists for FT_2 and FT_4 contain 536 and 545 variables respectively, but with only 40 and 34 unique variables respectively (about 93% overlap). Moreover, variable value captures program state only partially and at a coarse granularity. In Closure-11-69, following how SURE [54] and Reclues [55] establish distance using variable information (including both the involved variables and their values), the distance between FT_4 and FT_1 is 0.53 and that between FT_4 and FT_2 is 0.45, both of which are small and lower than that between FT_4 and FT_3 (0.57, still small) and that between FT_4 and FT_5 (0.56, still small). This means that if failures are indexed based on variable information, FT_4 would appear more similar to the $\{FT_1, FT_2\}$ group than to its true cluster, and thus would be likely to be grouped into the same cluster as FT_1 and FT_2 (SURE and Reclues exactly do this). Such misleading similarity relationships on top of variable information substantially weaken the discriminative power of program memory-based representations.

Limitations of Failure Clustering Methods. As described in Section 1, center-based clustering methods perform poorly for non-spherical clusters. We illustrate this limitation with Closure-1-3-4, which contains three faults, *i.e.*, Closure-1, Closure-3, and Closure-4. The three faults form three clusters with 8, 2, and 3 failing tests, respectively. Take SD-based failure representation method as an example, we utilize the Starcoder-3B model to encode the suspiciousness list generated by Dstar [68] for each failing test and apply t-SNE [40] to reduce the resulting features to two dimensions for visualization. As shown in Figure 1(b), the shape of the Closure-1 cluster is irregular. Center-based clustering methods cannot define an appropriate circular cluster boundary around the center of Closure-1 cluster: if the boundary is too small (*e.g.*, range-1), it fails to include the outliers at the top and bottom; if the boundary is too large (*e.g.*, range-2), it incorrectly encloses the data points of Closure-3 cluster. Thus, center-based clustering methods cannot handle this scenario. To conduct a further check, we follow Babaku et al. [10] and quantify cluster shape in the Defects4J-Multifault dataset using covariance matrices. We apply PCA (Principal Component Analysis) to obtain the eigenvalues along each dimension of the covariance matrix and compute the ratio between the minimum and maximum eigenvalues. If the ratio is close to 1, the cluster is more spherical; otherwise, it is more non-spherical. Excluding single-point clusters, the average ratio is 0.37 for clusters with at least two points, and drops to 0.12 for clusters with at least three points. This suggests that real-world failure clusters tend to be non-spherical in shape (especially when there are more failures), and center-based clustering methods thus are not well suited.

Summary. Therefore, a better failure indexing method is urgently needed. The new method must address two problems: (1) How to effectively capture deep failure information to distinguish failures with different causes? (2) How to effectively cluster failure clusters of various shapes?

4 Failure Semantic Graph

This section presents the Failure Semantic Graph (FSG), a novel graph-based failure representation method that effectively contains semantic information and runtime information of failures.

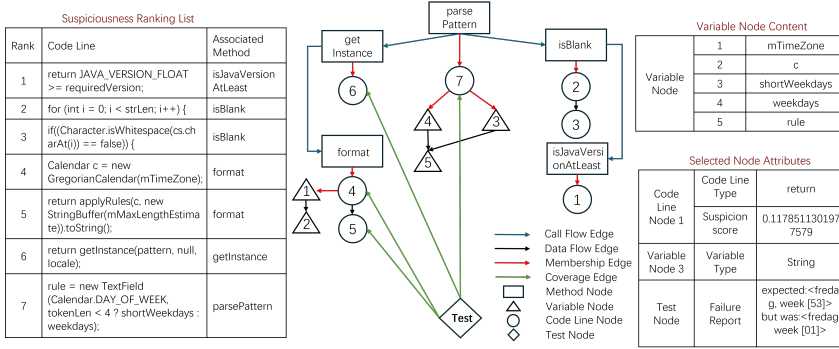


Fig. 2. An example FSG obtained by running the testLang645 test on a faulty Lang version with three faults.

4.1 Definition of the Graph

Recent advances in deep learning have spurred progress in code intelligence, where the quality of code representation plays a pivotal role [3]. Existing approaches fall into three main categories: token-based representations which offer simplicity but capture limited code semantic [2, 27]; syntax-based representations which leverage tree structures to capture syntactic hierarchy but often require significant refinement efforts due to their complexity [4, 47]; and semantic-based representations which employ graphs to model code behavior in a learnable way [11–13, 26]. A large amount of existing works [26, 38, 39, 58] have shown that graph-based code representations can effectively provide semantic information in addition to code sequences. Representative semantic representations include data flow graphs [13], control flow graphs [14], program dependence graphs [12], and contextual flow graphs [11], each capturing distinct aspects of program semantics.

In the field of failure indexing, previous CC-based and SD-based failure representation methods have shown that coverage information can provide certain insights about failure signature. However, being covered is a necessary but not sufficient condition for failure triggering according to the PIE model [65]. Thus, coverage information alone is not enough, and existing works have shown that CC-based and SD-based failure representation methods produce similar failure signature when dealing with multiple failures with a large amount of overlapping coverage [25, 34, 41]. Program memory information (including majorly variable values) partially encodes program state, and thus offers another perspective about failure signature compared to coverage information. However, variable value is a coarse-grained and incomplete characterization of program state. Given these drawbacks and the advantages of graph-based code representations, we design a novel graph representation called Failure Semantic Graph (FSG), which can capture the semantic information and runtime information of failures to distinguish failures of different root causes. In particular, FSG combines the coverage information and fine-grained data and call flow information in a compact way. For ease of reading, Figure 2 shows a simple FSG example.

Definition (Failure Semantic Graph) *The Failure Semantic Graph for a faulty program and a failed test is a triple $\langle V, E, X \rangle$ where V is a set of nodes, E is a set of directed edges between nodes in V , and X is a mapping from nodes in V to their attributes.*

We next elaborate on the structure of the graph. The node set V is categorized into four distinct types: the method node set V_m , the code line node set V_c , the variable node set V_v , and the test node set V_t . More specifically, the code line node set V_c represents the code lines in the suspiciousness ranking list generated after the run of the failed test and a certain fault localization technique. The method node set V_m represents the methods to which the code line from V_c belongs. The variable node set V_v corresponds to the set of involved variables in the code lines. To control the size of the graph, the variable node set does not include variables without data flow in the code line.

Specifically, a variable is excluded if it is not involved with any data-flow within a single statement (i.e., the data flow edges among variables in Figure 2). For example, given a code snippet “return a.m(b);” where a and b are variables and m is a function call, a and b will not be included in V_o since there is no data flow between a and b within this statement. The test node set V_t corresponds to the failed test used to run the faulty program.

Second, a directed edge in E can be of 4 kinds: call flow edge, data flow edge, membership edge, and coverage edge. Each edge connects the starting node V_1 to the ending node V_2 and can be represented as $\langle V_1, V_2 \rangle$. The call flow edge is between two method nodes ($V_1 \in V_m$ and $V_2 \in V_m$), denoting that method V_1 calls method V_2 . The data flow edge is between two variable nodes ($V_1 \in V_o$ and $V_2 \in V_o$) or two code line nodes ($V_1 \in V_c$ and $V_2 \in V_c$), representing that there is a data flow between the two nodes. The membership edge connects the method node to the code line node ($V_1 \in V_m$ and $V_2 \in V_c$) or the code line node to the variable node ($V_1 \in V_c$ and $V_2 \in V_o$), representing that a code line belongs to a method or a variable belongs to a code line. The coverage edge connects the test node to the code line node ($V_1 \in V_t$ and $V_2 \in V_c$), representing that the test has executed the corresponding code line.

Finally, the mapping X assigns specific attributes to certain nodes in the set V , with the attribute types varying according to the category of the node. Currently, we add special attribute information to code line nodes, variable nodes, and test nodes. For code line nodes, we use the code line type and the suspicion score of code line as their attributes. For variable nodes, we use the type of the variable as their attributes. For test nodes, we use the failure reports of tests as their attributes.

Overall, FSG is a directed, labeled graph that captures deep semantic information and runtime information of failures in a compact way. Unlike CC- and SD-based representations that rely only on coverage information and program memory-based representations that focus primarily on variable information, FSG particularly captures fine-grained data-flow and call-flow information among program entities along with the coverage information of failing tests. In Closure-11-69 (Figure 1(a)), f_1 is located at lines 1314–1315 in a conditional branch statement, while f_2 is located at line 1581 in a method-invocation statement. To distinguish failures due to different faults, FSG makes these distinctions explicit by (1) linking each suspicious line to its enclosing method via membership edges (e.g., f_1 in `visitGetProp` and f_2 in `visitCall`), (2) connecting statements via fine-grained data-flow among the involved variables (e.g., the line 1557 $\rightarrow$ line 1581 for f_2 based on `functionType`), (3) capturing the call flow edges when suspicious code spans multiple methods, and (4) capturing distinct node kinds and their attribute information (e.g., code line node for line 1314 in f_1 with the suspiciousness value 0.33 and code line node for line 1581 in f_2 with the suspiciousness value 0.25). These enable FSG to represent failure information more comprehensively, thereby distinguishing failures with different causes more effectively.

4.2 Construction of the Graph

We next present the workflow for constructing FSG. The workflow takes a given faulty program P and a test suite T as inputs, and outputs the corresponding FSG.

Step 1: Generating coverage information and suspiciousness ranking lists. We first use the test suite T to execute program P and record the test results and coverage information. For a test, if the result of executing the program on it meets the expected result, we consider it a successful test T_s ; otherwise, we consider it a failed test T_f . For each failed test, we record the code lines that it covers and represent them as coverage edges ($\langle T_f, V_c \rangle \in E$) in FSG. Based on the coverage information for all tests in T , following the two strong baselines SURE [54] and ReClues [55], we use the widely used DStar formula [68] to generate a suspiciousness ranking list for each failed test.

Step 2: Extracting membership relationship. For the ranking list L_i corresponding to a failed test, we match the code lines in the list with the program content. The matched content includes

the file F_i where the code is located and the method V_m where the code is located. Then, we build a membership edge ($< V_m, V_c > \in E$) between the associated method and the statement code.

Step 3: Extracting call flow and data flow relationships. For the code content corresponding to the line in the suspiciousness ranking list, we use static analysis to generate data flow and get variable attributes. For the associated methods, we use static analysis to generate call flow. Our notions of data flow and control flow are standard, and are in line with typical data flow analysis and call flow analysis. Based on the static analysis results, we build data flow edges ($< V_v, V_v > \in E$ and $< V_c, V_c > \in E$), call flow edges ($< V_m, V_m > \in E$), and also the membership edges ($< V_c, V_v > \in E$) between variable nodes and method nodes.

Handling indirect dependencies. To make the analysis accurate, we explicitly account for indirect dependencies that arise due to indirect calls (e.g., Java dynamic dispatch and C function pointers) and aliasing. In order for keeping the analysis affordable, we adopt lightweight, type-driven approximations and add the resulting relations in the FSG. Specifically, we resolve Java dynamic dispatch via Rapid Type Analysis (RTA) [57], keeping only call targets from instantiated, type-compatible receiver classes. We also resolve C function-pointer calls via type matching (also known as signature matching) [77], yielding call target functions whose signatures are compatible with the function-pointer type. Each resolved target is added as a call-flow edge ($< V_m, V_m > \in E$). For aliasing, following the prior scalable analysis [18], we approximate aliasing relations using type compatibility together with high-level information such as field names. If two variables are potentially aliased under this approximation, we accordingly add the related data-flow edges $< V_v, V_v > \in E$ and/or $< V_c, V_c > \in E$ in the FSG.

Step 4: Constructing the failure semantic graph. We use the failure semantic graph to integrate the information generated in the previous steps. It uses a sequence list that includes code method information, code line information, variable information, failed test information, and an adjacency matrix to record node relationships (membership relationship, call flow relationship, data flow relationship, and coverage relationship).

Note that during the construction of FSG, different ranges of the suspiciousness ranking list can be selected and we will explore the impact of different selection strategies in detail in our evaluation. In addition, if a certain node is not connected with any other nodes after the above 4 steps (i.e., isolated node), the node is deleted from the final FSG.

5 GREClue Approach for Failure Indexing

This section introduces the full GREClue approach, a novel failure indexing technique with graph-based failure representation and entropy-based deep clustering. GREClue can cluster failed tests without knowing the number of faults and presetting cluster centers.

5.1 Overview

While the proposed failure graph representation (FSG) can effectively capture semantic information and runtime information of failures, we further need to effectively apply the FSG to the clustering process to achieve failure indexing. To solve the problem of diverse cluster shapes and unknown number of faults in the failure indexing task, we propose the GREClue approach. Figure 3 shows the process of GREClue. Given a faulty program and a test suite, the workflow of GREClue is as follows: a) First, we preprocess a faulty program and a test suite to obtain the FSGs and suspiciousness ranking lists. Note that each failing test generates a corresponding FSG and a corresponding suspiciousness ranking list; b) Then, we obtain the graph features of FSGs and the sequence features of the suspiciousness ranking lists through GNN and code language models respectively; c) Next, we use the entropy-based deep clustering method to cluster the failures according to the generated failure features; d) Finally, the results of failure clustering by GREClue are evaluated.

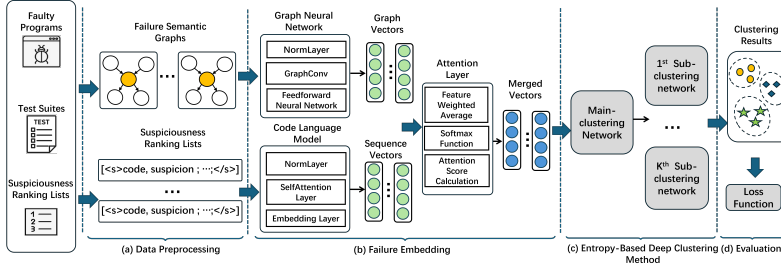


Fig. 3. An overview of the GREClue Approach.

5.2 Failure Embedding

5.2.1 Embedding for Sequence Features. We in this study regard the suspiciousness ranking list, which contains code lines and the suspicion scores, as the failure sequence information. To obtain the failure sequence feature, we leverage code language models built by the stacked transformer [63], which is the most widely adopted approach in software engineering [29, 42, 78]. These models can convert the suspiciousness ranking list into a feature vector that will be used in clustering models. Given a failure suspiciousness ranking list X , where X_i denotes the i -th line in the suspiciousness ranking list, including the code line C_i and the suspicion score S_i . The failure sequence information is represented as:

$$X = \{C_1, S_1; \dots; C_m, S_m; \dots; C_n, S_n\} \quad (1)$$

Then, the failure sequence information is converted into a fixed-dimensional vector representation through the code language models. The failure sequence vector representation is:

$$E_{seq} = \text{Embedding}(X) = \{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n\} \quad (2)$$

where E_{seq} represents the failure sequence feature vector and $\mathbf{e}_i$ is the feature vector of the i -th line in the suspiciousness ranking list.

5.2.2 Embedding for Graph Features. To enrich the failure feature vector, we need to obtain additional feature of the FSG. Following the procedure in Section 4, we can construct the FSG and extract the node matrix N , adjacency matrix M , and attribute matrix A . To effectively obtain the graph feature of FSG, we first use the code language models to map the FSG content to the same vector space as the sequence feature vector, and then use the graph neural network (GNN) [51] to extract the graph feature of FSG. We fuse the sequence feature of each failure and its corresponding graph feature into a merged failure feature.

First, we process the nodes and their attributes in the FSG. We use the code language models to extract node and attribute features. To integrate the node attributes with the graph information, we fuse node attribute features into their corresponding node feature. The detailed steps are as follows:

$$F_N = \text{Embedding}(N) \quad F_A = \text{Embedding}(A) \quad (3)$$

$$H_n = \text{concat}(F_N, F_A) \quad F = \text{Linear}_1(H_n) \quad (4)$$

where Linear_1 is a feed-forward network layer used to change the node feature dimension. Following this process, we get new node features F .

Next, we extract the feature of the graph formed by the nodes. To do so, we apply graph convolution to aggregate the node features within the graph. By propagating the node features, graph convolution effectively captures the graph feature. The detailed steps are as follows:

$$\text{GraphConv}(F, M) = \sigma \left(D^{-1/2} M D^{-1/2} F W \right) \quad D = \sum_{i=1}^n M_i \quad (5)$$

$$E_{graph} = GraphConv(F, M) \quad (6)$$

where E_{graph} is the failure graph feature, D is the degree matrix of the variable node, W is the weight matrix, and σ is the nonlinear activation function.

Finally, we use the attention mechanism to fuse the sequence feature and graph feature of each failure. This process can be expressed as follows:

$$F_{failure} = Attention(E_{seq}, E_{graph}) \quad (7)$$

5.3 Entropy-Based Deep Clustering Method

Based on the acquired failure features, we need to cluster the failures according to the triggering causes without knowing the number of faults. To achieve this, we propose an entropy-based deep clustering method, which aims to minimize the diversity within clusters and maximize the diversity between clusters by merging and splitting clusters. We use the entropy between data points to measure the data diversity within and between clusters. The clustering process is divided into the following steps: a) First, we initialize the number of clusters and set it to the same as the number of failed tests. This means that each failed test is a cluster at the beginning. We use K to denote the set of clusters and N_K to denote the number of clusters. Note that N_K is variable; b) Then, when the difference between clusters is small, we merge two similar clusters; c) Next, when the intra-cluster diversity is large, we split the cluster into two subclusters; d) After multiple rounds of cluster merging and splitting, the number of clusters will no longer change and actually is the number of faults predicted by the model. Figure 4 shows the process of the entropy-based deep clustering method, which does not need to preset the cluster centers.

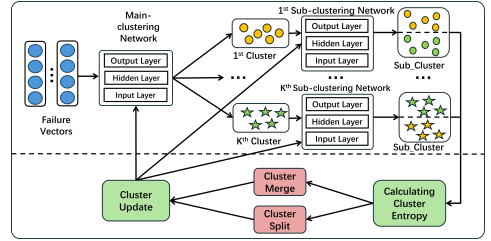


Fig. 4. An overview of the Entropy-Based Deep Clustering Method.

5.3.1 Model Structure. Our clustering model consists of two parts: a main-clustering network and N_K sub-clustering networks (corresponding to the number of clusters). Each clustering network is composed of a feedback neural network with input layer, hidden layers, and output layer, which can be expressed as:

$$F_h = hidden(W_h input(F_{failure}) + W_x F_{failure} + b_1) \quad (8)$$

$$F_o = output(W_o F_h + b_2) \quad (9)$$

where *input*, *hidden*, and *output* are all fully connected layers, W_h , W_x , and W_o are the weight matrixs, b_1 and b_2 are bias. The output dimension of the main-clustering network is the number of clusters N_K , and the output dimension of the sub-clustering network is 2.

Given a set of failure feature vectors, the vectors are first input into the main-clustering network to generate the probability P_i^m of belonging to the i -th cluster. We select the cluster with the highest probability as the cluster C to which the failure belongs. The specific operation is expressed as:

$$(P_i^m)_{i=1}^{N_K} = Main_Cluster(F_{failure}) \quad \sum_{i=1}^{N_K} P_i^m = 1 \quad (10)$$

$$C = argmax_i P_i^m \quad (11)$$

Then, according to the results of the main-clustering network, the failure features are input into the sub-clustering networks corresponding to the clusters to which they belong. The sub-clustering network will output two probabilities to further divide the data into two subclusters. The purpose

of this process is to facilitate the subsequent cluster splitting. Note that when the sub-clustering network is fed with only one data point, no subcluster is generated. The steps can be expressed as:

$$(P_{k,j}^s)_{j=1}^2 = \text{Sub_Cluster}(F_{failure}^k) \quad \sum_{j=1}^2 P_{k,j}^s = 1 \quad (12)$$

where $F_{failure}^k$ represents the fault feature that is classified into the k -th sub-cluster network, and $P_{k,j}^s$ represents the probability that fault $F_{failure}^k$ belongs to the j -th sub-cluster.

5.3.2 Cluster Merge. The goal of cluster merging is to merge two similar clusters into one cluster. We use entropy to measure the differences between data points. A lower entropy means that the data points are more similar and should be in one cluster. Conversely, a higher entropy means that the data points are more dispersed and should be in different clusters. The entropy value of the i -th cluster K_i can be expressed as:

$$p(x_i, x_j) = \text{norm}(\exp(-\frac{d(x_i, x_j)^2}{2\sigma^2})) \quad (13)$$

$$H(K_i) = - \sum_{i=1}^n \sum_{j=1, j \neq i}^n p(x_i, x_j) \log p(x_i, x_j) \quad (14)$$

where $H(K_i)$ represents the entropy value of the i -th cluster, $d(x_i, x_j)$ represents the Euclidean distance between data points, σ represents the standard deviation of the data point distance, and norm is the normalization function.

We consider the change in entropy after merging two clusters as the cost of the merge operation ΔH_m . If the cost of merging two clusters is much smaller than that of merging any other two clusters, we believe that the merged cluster will be pure and will not cause information confusion. On the contrary, if the cost is large, we believe that the degree of confusion within the merged cluster is high and the merger is not worthwhile. The merging condition can be expressed as:

$$\Delta H_m(K_1, K_2) = H(K_1) + H(K_2) - H(K_1 \cup K_2) \quad (15)$$

$$2\lambda_1 \Delta H_m(K_1, K_2) \leq \frac{1}{N_K(N_K - 1)} \sum_{i=1}^{N_K} \sum_{j=1, i \neq j}^{N_K} \Delta H_m(K_i, K_j) \quad (16)$$

where λ_1 is a hyperparameter used to control the threshold of cluster merging conditions. We perform a grid search over $\lambda_1 \in \{1, 2, 3, 4, 5\}$ on the training projects and select $\lambda_1 = 3$ as it achieves the best validation performance. K_1 and K_2 represent the two clusters to be merged.

In terms of technical implementation, if clusters K_i and K_j are merged, we change the dimension of the output layer of the main-clustering network to $N_K - 1$ by mean pooling. At the same time, the weights of the two sub-clustering networks corresponding to K_i and K_j are averaged to generate the new sub-clustering network weight of the merged cluster.

5.3.3 Cluster Split. The goal of cluster splitting is to split a cluster with very different data points into two clusters. When the data entropy value in a cluster is too high, it means that the data points in the cluster are very different. Similar to cluster merging, we consider the change in entropy after a cluster split as the cost of the split operation ΔH_s . If the cost of splitting a cluster is much smaller than the cost of splitting any other cluster, we believe that the cluster should be split. The cluster splitting condition can be expressed as:

$$\Delta H_s(K_i) = H(K_i) - H(K_{i,1}) - H(K_{i,2}) \quad (17)$$

$$\lambda_2 \Delta H_s(K_i) \leq \frac{1}{N_K - 1} \sum_{k_1=1}^{N_K} \Delta H_s(K_{i,1}, K_{i,2}) \quad (18)$$

where λ_2 is a hyperparameter used to control the threshold of cluster splitting conditions. Similar to λ_1 , we search over $\lambda_1 \in \{1, 2, 3, 4, 5\}$ and choose $\lambda_2 = 4$, which yields the best validation performance. $K_{i,1}$ and $K_{i,2}$ are two subclusters generated after splitting.

For technical aspects, if cluster K_i is to be split into two clusters $K_{i,1}$ and $K_{i,2}$, we first obtain the two subclusters generated by the sub-clustering network corresponding to K_i . Then, we copy the sub-clustering network weights of K_i to build a new sub-clustering network for the increase in the number of clusters. In the main-clustering network, we expand the dimension of the output layer to $N_K + 1$ and copy the output weights of K_i to the newly added output weights of K_{N_K+1} .

Unlike the center-based clustering methods in prior failure indexing works, entropy-based deep clustering does not preset cluster centers. It instead evaluates cluster quality using entropy, which assesses cluster quality by measuring the diversity of data within clusters of arbitrary shapes. When the entropy value between two clusters is small, it merges them into a single cluster through a merge operation. When the entropy value within a cluster is large, it splits that cluster into two sub-clusters through a split operation. In Closure-1-3-4 (Figure 1(b)), the method starts from an initialization where each point (*i.e.*, failure) forms a singleton cluster, and then performs entropy-guided merges that progressively aggregate the Closure-1 points. As the merges proceed, Closure-1 points are first consolidated into two coarse-grained sub-clusters (roughly corresponding to the upper and lower parts of the curved distribution), and these sub-clusters are further merged into a coherent yet non-spherical Closure-1 cluster. During this coarse aggregation, the nearby Closure-3 points on the left may be temporarily absorbed into the same cluster. However, doing this increases the intra-cluster entropy. Thus, the algorithm triggers a split operation to decompose the mixed cluster and separate Closure-1 points from Closure-3 points according to their differing distributions. As a result, entropy-based deep clustering effectively identifies irregularly shaped failure clusters without presetting cluster centers.

5.4 Training and Inference

To adapt to the real situation of failure indexing task, the GREClue approach uses the unsupervised training method, which uses unlabeled data during the training process. We train and test GREClue using two of the most popular datasets in failure indexing: SIR and Defects4J-Multifault (see Sec 6.2 for details). Specifically, to ensure a fair comparison, our training setting is consistent with the state-of-the-art failure indexing approach SURE [54], which uses 30% of the SIR dataset for training and 70% of the SIR dataset as well as Defects4J-Multifault dataset for testing. The baseline uses this setting to gain greater confidence in the applicability of the approach, and using more training data is likely to further improve performance. To account for randomness, following the guideline in [9], we randomly split the SIR dataset according to the 30-70 ratio 30 times and the reported result of GREClue is the average of the 30 results obtained for these splits. After the model completes a clustering, we can obtain the probabilities P_i^m and $P_{k,j}^s$ generated by the main clustering network and the sub-clustering networks. Referring to the training method of previous clustering works [48, 59, 61], we use the posterior probabilities of P_i^m and $P_{k,j}^s$ as the pseudo labels required to calculate the loss. We denote the posterior probability of P_i^m as E_i^m and the posterior probability of $P_{k,i}^s$ as $E_{k,j}^s$, respectively. We use KL divergence to calculate the loss. The total loss $\mathcal{L}_{total}$ is the sum of the main-clustering network loss $\mathcal{L}_{main}$ and the loss $\mathcal{L}_{sub}$ of multiple sub-clustering networks. The specific operations are as follows:

$$\mathcal{L}_{main} = \sum_{i=1}^{N_K} \text{KL}(P_i^m \| E_i^m) \quad (19)$$

$$\mathcal{L}_{sub} = \sum_{k=1}^{N_k} \sum_{j=1}^{I_k} \text{KL}(P_{k,j}^s \| E_{k,j}^s) \quad (20)$$

$$\mathcal{L}_{total} = \mathcal{L}_{main} + \mathcal{L}_{sub} \quad (21)$$

During inference, we first construct FSGs according to the procedure in Section 4. Then, we use the trained GNN and code language models to encode the FSGs and suspiciousness ranking lists to obtain failure graph features and failure sequence features, and combine failure graph features and sequence features to obtain new failure features. Next, the trained GREClue will cluster the failures according to the failure features. Finally, the correctness of the clustering results is evaluated.

6 Experimental Setup

6.1 Research Questions

For reasonably analyzing our approach, we explore the following research questions:

RQ1 (Parameter Analysis): How parameter values affect the performance of our approach? During the construction of FSG, different ranges of the suspiciousness ranking list can be selected. Thus, we vary the selection ranges to evaluate the performance of GREClue in different scenarios.

RQ2 (Effectiveness): How does our approach perform compared to other existing works on the failure indexing task? To pursue this question, we evaluate the approach on two widely used datasets and compare the performance with that of the state-of-the-art failure indexing methods.

RQ3 (Impact Analysis): How much influence does each component of the approach have on the final result? We gradually remove components from the approach to evaluate the contribution of each component.

RQ4 (Parallel Debugging Effectiveness): Can the failure clustering results generated by GREClue effectively help with parallel debugging? We apply the results of GREClue to the parallel debugging scenario to demonstrate the usefulness of GREClue.

6.2 Datasets

Defects4J-Multifault. Defects4J [32] is a dataset consisting of multiple real-world Java projects and has been widely used in the field of software testing and debugging [71, 75, 76]. Defects4J is originally designed for single-fault scenarios, where the provided test suite is designed to reveal only one fault in the program, regardless of how many faults exist. Recent failure indexing works [53–55] have created 100 Defects4J faulty version with the number of faults ranging from 1 to 5, and we use this dataset to facilitate experimentation and result comparison. Among the 100 faulty versions, there are 15, 35, 25, 16, and 9 versions with 1, 2, 3, 4, and 5 bugs, respectively. The dataset is based on five Java projects from Defects4J (Table 1 describes them in detail), and is created by searching for 1-bug, 2-bug, 3-bug, 4-bug, and 5-bug faulty versions according to the search strategy proposed by An et al. [6]. The search strategy involves enhancing the test suite by transferring failure-revealing tests from other faulty versions to the original version.

Table 1. Datasets used in our experiment.

	Project	Version	kLOC	Functionality
C	flex	2.5.3	14.5	Parser generator
	grep	2.4	13.5	Text matcher
	gzip	1.2.2	7.3	File archiver
	sed	3.02	10.2	Stream editor
Java	Chart	2.0.0	96.3	Chart library
	Closure	2.0.0	90.2	Closure compiler
	Lang	2.0.0	22.1	Apache commons-lang
	Math	2.0.0	85.5	Apache commons-math
	Time	2.0.0	28.4	Date and time library

Software-artifact Infrastructure Repository (SIR).

SIR is an open-source dataset [19] commonly used in software engineering research, and it contains multiple software projects and their related tests, failures, versions, and other metadata. Using four widely used C projects (see Table 1 for details), recent failure indexing works [53, 54] have created SIR faulty versions with the number of faults ranging from 1 to 5. These faulty versions are created by mutation-based fault injection [43]. Note that previous studies have confirmed that mutation-based

faults can be a reliable proxy for real-life faults in experiments conducted by software testing and debugging research [7, 8, 47]. To keep the experimental environment consistent, we again directly

use this created SIR dataset. Among the 1000 SIR faulty versions, the number of versions with 1 bug is 40, and the number of versions with 2, 3, 4, and 5 bugs is 240 each.

6.3 Baselines

We select eight state-of-the-art failure indexing approaches as the baselines, and they are as follows: **CC-based**. For approaches that use CC-based failure representation, we select Cov_{hit} [28] as the baseline for its popularity. Cov_{hit} represents failures using coverage information of binary indicators, and we consider three versions of Cov_{hit} to make the baseline comparison strong. The considered three versions include $\text{Cov}_{\text{hit-Cos}}$, $\text{Cov}_{\text{hit-Jacc}}$, and $\text{Cov}_{\text{hit-Eucl}}$, and they correspond to using the Cosine distance, the Jaccard distance, and the Eculidean distance for handling binary vectors respectively. In addition, we consider another baseline $\text{Cov}_{\text{Count}}$ [52, 66] which represents failures using coverage information of execution frequency.

SD-based. For approaches that use SD-based failure representation, we select MSeer [22] as the baseline. MSeer is the state-of-the-art technique that uses SD-based failure representation, and it uses Crosstab [69] to produce the suspiciousness ranking list and uses an improved Kendall tau distance metric to calculate the distance. A variant $\text{MSeer}_{\text{EFF}}$ [54] uses a risk evaluation formula based on genetic programming to optimize the suspiciousness ranking list.

ReClues. ReClues [55] obtains the runtime values of certain program variables (by setting breakpoints) to represent failures, and accordingly designs rules to measure their similarity.

SURE. SURE [54] is a failure indexing method based on the program memory spectrum. It collects memory information (including variable values and stack frame depth) at preset breakpoints and converts it into an image, which is then converted into the failure feature through a CNN.

For results about the baselines on the studied datasets, we directly adopt the results published in the corresponding papers to promote fair comparison.

6.4 Metrics

To assess the results, we account for multiple evaluation metrics. First, we need to evaluate the extent to which the number of faults can be correctly predicted. To achieve this, we use the metric V_{equal} , which represents the number of faulty versions for which an approach correctly predicts the number of faults. Second, we need to evaluate the clustering effectiveness, *i.e.*, the extent to which the failures can be indexed according to the root cause. To achieve this, we use the following metrics: the Fowlkes and Mallows Index (FMI), the Jaccard Coefficient (JC), the Precision Rate (PR), the Recall Rate (RR), and the F1 score ($F1$). Among them, FMI and JC are used to compare

the consistency of each pair of failed tests in the generated cluster with that in the oracle cluster, and can be calculated as $FMI = \sqrt{X_{SS}/(X_{SS} + X_{SD}) * X_{SS}/(X_{SS} + X_{DS})}$ and $JC = X_{SS}/(X_{SS} + X_{SD} + X_{DS})$. Here, the subscript (*i.e.*, SS and SD) denotes the possible scenarios in the comparison as shown in Table 2, and the constituent parts like X_{SS} denote the number of pairs of “ SS ”. PR , RR , and $F1$ describe whether the classification result of each failed test in the generated cluster is consistent with that in the oracle cluster, and can be calculated as $PR = X_{TP}/(X_{TP} + X_{FP})$, $RR = X_{TP}/(X_{TP} + X_{FN})$, and $F1 = 2 * (PR * RR)/(PR + RR)$. Here, the subscript (*i.e.*, TP and FP) again denotes the possible scenarios in the comparison as shown in Table 2, and the constituent parts like X_{TP} denote the number of failures of “ TP ”. Similar to the experimental setting of baselines [54, 55], which only

Table 2. Scenarios in two types of metrics.

Metric	Notation	Result of failure indexing	
		In generated cluster	In oracle cluster
FMI JC	SS	Same	Same
	SD	Same	Difference
	DS	Difference	Same
	DD	Difference	Difference
PR RR	TP	Positive	Positive
	FP	Positive	Negative
RR	TN	Negative	Negative
	FN	Negative	Positive

the number of failures of “ TP ”. Similar to the experimental setting of baselines [54, 55], which only

Table 3. Comparison in different selection ranges.

Model	V_{equal}	S_M_{FMI}	S_M_{JC}	S_M_{PR}	S_M_{RR}	S_M_{F1}
Unique Data+Top-10	33	32.51	32.03	30.08	29.79	29.93
Unique Data+Top-50	39	38.63	38.21	36.01	35.67	35.83
Unique Data+Top-100	44	43.72	43.46	40.07	39.29	39.67
Unique Data+Top-200	50	49.35	49.03	46.58	46.19	46.38
Unique Data+ALL	41	40.26	39.94	37.72	37.25	37.48

evaluates the clustering effectiveness for faulty versions whose numbers of faults are correctly predicted (for simplicity, referred to as “ $k == r$ ” faulty versions hereafter). Accordingly, we use the formula $S_M_M = \sum_i^{V_{equal}} M_i$ to calculate the sum of the metric values. Here, S_M is the abbreviation for “Sum_Metrics”, V_{equal} is the number of “ $k == r$ ” faulty versions, and M_i denotes the value of the clustering metric M for the i -th “ $k == r$ ” faulty version (M takes FMI , JC , PR , RR , or $F1$).

Note that the evaluation metrics V_{equal} and S_M_M are also used by the recent baselines [54, 55] to evaluate the failure indexing performance.

6.5 Implementation Details

To establish FSG for Java and C code, we respectively use Spoon [44] and Tree-sitter [23] to conduct the static analysis. The learning part of our approach is implemented using the Pytorch [49] framework and it uses the Adam optimizer [33] to update the model parameters. As the training process progresses, the learning rate is adjusted (ranging from 0 to 0.0005) to adapt to the model learning at different stages. We set the batch size of the model to 10 and the epoch to 300. Our evaluations are performed on an Ubuntu 22.04.5 server, featuring a 64-core Intel Xeon Gold 6226R CPU and two RTX A6000 GPUs that can update the 3B parameters of code language models.

7 Experimental Result

7.1 (RQ1) Parameter Analysis

We studied the impact of different selection ranges of the suspiciousness ranking list on the results. To highlight the differences between failures, we first select the contents of one ranking list that are different from other suspiciousness ranking lists as the selection range. Second, to obtain more failure information, we add the top-50, top-100, top-200, and all code lines of the ranking list to the selection range and evaluate the results of GREClue. Table 3 shows the results of Codebert-based GREClue (i.e., use Codebert as the code language model) on the Defects4J-Multifault dataset.

From the data in the table, we can see that when the selection range is the unique data and the top-200 data of the suspiciousness ranking list, the performance of Codebert-based GREClue is the best. Based on the top-200 data, GREClue correctly predicts the number of faults for 50 faulty versions. In terms of clustering results, the FMI , JC , PR , RR , and $F1$ of GREClue are 49.35, 49.03, 46.58, 46.19, and 46.38, respectively. By analyzing the results, we believe that the top-10 and top-50 data contain incomplete semantic and runtime information, and thus it is difficult to distinguish different failures based on these data. The top-100 data can obviously help the model distinguish different failures, but the amount of effective information contained is still small. Using all the data brings a lot of noisy that affects the performance of the approach.

Answer to RQ1: The experimental results show that when the selection range of the suspiciousness ranking list is the unique data plus the top-200 data, GREClue has the best performance. Thus, in subsequent experiments, we uniquely use this range.

7.2 (RQ2) Model Effectiveness

To ensure the fairness of the experiment, we first use Codebert as the code language model to keep the same parameter level as the baseline models (in particular the state-of-the-art approach SURE

Table 4. Effectiveness comparison on Defects4J-Multifault and SIR datasets.

Model	Datasets											
	Defects4J-Multifault						SIR					
	V_{equal}	$S_{M_{FMI}}$	$S_{M_{JC}}$	$S_{M_{PR}}$	$S_{M_{RR}}$	$S_{M_{F1}}$	V_{equal}	$S_{M_{FMI}}$	$S_{M_{JC}}$	$S_{M_{PR}}$	$S_{M_{RR}}$	$S_{M_{F1}}$
Cov _{hit} -Cos	16	15.97	15.97	15.30	15.15	15.22	27	19.76	18.77	18.90	11.53	14.32
Cov _{hit} -Jacc	19	19.00	18.75	18.61	18.59	18.59	32	24.97	21.02	22.52	13.67	17.01
Cov _{hit} -Eucli	20	20.00	20.00	20.00	20.00	20.00	32	25.31	21.34	23.18	14.92	18.15
Cov _{count}	24	23.98	23.96	23.50	23.50	23.50	54	43.33	36.75	42.37	31.03	35.82
MSeer	29	28.99	28.98	28.75	28.75	28.75	83	62.86	51.72	63.08	39.26	48.39
MSeer _{EFF}	33	32.95	32.86	32.25	32.10	32.17	133	104.38	86.98	105.52	74.04	87.02
ReClues	37	36.92	36.85	35.92	36.12	36.02	152	199.07	99.58	111.76	77.52	91.54
SURE	41	40.81	40.63	37.64	37.44	37.53	167	131.15	109.70	123.10	80.44	97.29
GREClue-Codebert(our)	50	49.35	49.03	46.58	46.19	46.38	183	145.24	121.73	135.56	88.59	107.15
GREClue-Starcoder3B(our)	58	57.66	57.27	52.65	52.04	52.34	200	157.11	132.64	151.41	95.22	116.91

Table 5. Statistical tests on Defects4J-Multifault and SIR datasets.

Model	Datasets		V_{equal}	$S_{M_{FMI}}$	$S_{M_{JC}}$	$S_{M_{PR}}$	$S_{M_{RR}}$	$S_{M_{F1}}$
SURE	Defects4J-	p -value	0.000168	0.000172	0.000172	0.000172	0.000172	0.000172
+	Multifault	A_{12} -test	1.0	1.0	1.0	1.0	1.0	1.0
GREClue-Codebert	SIR	p -value	0.000171	0.000173	0.000173	0.000173	0.000173	0.000173
		A_{12} -test	1.0	1.0	1.0	1.0	1.0	1.0
SURE	Defects4J-	p -value	0.000167	0.000169	0.000169	0.000169	0.000169	0.000169
+	Multifault	A_{12} -test	1.0	1.0	1.0	1.0	1.0	1.0
GREClue-Starcoder3B	SIR	p -value	0.000169	0.000170	0.000170	0.000170	0.000170	0.000170
		A_{12} -test	1.0	1.0	1.0	1.0	1.0	1.0

[54]). To improve the performance of the model as much as possible, we also use Starcoder3B as the code language model. We compare GREClue with the selected baselines on Defects4J-Multifault dataset and SIR dataset. Since the experimental setting of ReClues [55] on the SIR dataset is different from other baselines, we replicate Reclues according to its documentation and use 70% of the SIR data to see its performance as other baselines. We replicate the selection of the SIR data 30 times and use the average as the final result for ReClues on the SIR dataset.

The left half of Table 4 shows the results of failure indexing models on the Defects4J-Multifault dataset. From the table, we can see that the results of GREClue are better than all baselines. In terms of predicting the number of faults, Codebert-based GREClue and Starcoder-based GREClue correctly predict the number of faults for 50 faulty versions and 58 faulty versions, respectively. Compared with the results of Cov_{hit}-Cos, Cov_{hit}-Jacc, Cov_{hit}-Eucli, Cov_{hit}-Count, MSeer, MSeer_{EFF}, ReClues, and SURE, the number of faulty versions correctly predicted by Starcoder-based GREClue increases by 263%, 205%, 190%, 142%, 100%, 76%, 57%, and 41%, respectively. In terms of clustering results, compared with the results of Cov_{hit}-Cos, Cov_{hit}-Jacc, Cov_{hit}-Eucli, Cov_{hit}-Count, MSeer, MSeer_{EFF}, ReClues, and SURE, the Starcoder-based GREClue improves FMI , JC , PR , RR and $F1$ by 41% to 261%, 41% to 259%, 40% to 244%, 39% to 243%, and 39% to 244% respectively.

The right half of Table 4 shows the results of failure indexing models on the SIR dataset. From the table, GREClue still outperforms all baselines. In terms of predicting the number of faults, Codebert-based GREClue and Starcoder-based GREClue correctly predict the number for 183 and 200 faulty versions respectively. Compared with the results of Cov_{hit}-Cos, Cov_{hit}-Jacc, Cov_{hit}-Eucli, Cov_{hit}-Count, MSeer, MSeer_{EFF}, Reclues, and SURE, the number of faults correctly predicted by Starcoder-based GREClue increases by 641%, 525%, 525%, 270%, 141%, 50%, 32%, and 20%, respectively. In terms of clustering results, compared with the results of Cov_{hit}-Cos, Cov_{hit}-Jacc, Cov_{hit}-Eucli, Cov_{hit}-Count, MSeer, MSeer_{EFF}, Reclues, and SURE, the Starcoder-based GREClue improves FMI , JC , PR , RR and $F1$ by 20% to 695%, 21% to 607%, 23% to 701%, 18% to 726%, and 20% to 716%, respectively.

To determine the statistical significance of the GREClue advantage, we select two tests following the guideline in [9]: 1) the Mann-Whitney-Wilcoxon test [67] to determine the statistical significance of the results, and 2) the A_{12} value [62] statistics to determine the strength of the significance. To conduct the test, given the original SURE paper runs the best baseline SURE only one time, we

Table 6. Ablation Study on Defects4J-Multifault and SIR datasets.

Model	Datasets											
	Defects4J-Multifault						SIR					
	V_{equal}	S_{FMI}	S_{MC}	S_{MPR}	S_{MRR}	S_{MF1}	V_{equal}	S_{FMI}	S_{MC}	S_{MPR}	S_{MRR}	S_{MF1}
GREClue-Codebert	50	49.35	49.03	46.58	46.19	46.38	183	145.24	121.73	135.56	88.59	107.15
GREClue-Suspicion_List	42	41.65	41.31	38.29	37.82	38.05	171	132.08	106.54	123.85	80.18	97.34
GREClue-K-medoids	35	34.70	34.43	33.01	32.71	32.85	162	125.93	101.41	117.34	76.21	92.40
GREClue-RMALL	32	31.67	31.42	29.96	29.63	29.79	154	116.98	94.75	113.46	73.58	89.27
GREClue-Starcoder3B	58	57.66	57.27	52.65	52.04	52.34	200	157.11	132.64	151.41	95.22	116.91
GREClue-Suspicion_List	52	51.67	51.33	48.59	48.29	48.43	190	147.72	119.14	139.96	88.02	108.07
GREClue-K-medoids	44	43.77	43.46	40.36	40.15	40.25	182	137.93	113.21	134.17	84.28	103.53
GREClue-RMALL	39	38.57	38.26	35.78	35.34	35.55	173	109.13	108.65	127.53	80.23	98.49

replicate SURE according to its documentation and inquire the authors for relevant configurations to make sure our run is consistent with the SURE paper. Then, we run SURE 30 times using exactly the same training and testing data as given in Sec 5.4. The results are shown in Table 5. In our experiments, the test results can be interpreted as follows: If the Mann-Whitney-Wilcoxon p -value is smaller than 0.05, it means that there is a significant difference between the results generated by GREClue and SURE. The A_{12} statistical test indicates which algorithm is most likely to be better. If A_{12} is equal to 0.5, it means that there is no difference between GREClue and SURE. If A_{12} is greater than 0.5, it means that GREClue is more likely to produce better results than SURE, and vice versa. We can see from the results that on the Defects4J-Multifault and SIR datasets, the p -values for all metrics are much smaller than 0.05, indicating that there exist significant differences between the results of GREClue and that of SURE. The A_{12} value is 1.0 for all metrics on both datasets, indicating that the proposed GREClue approach significantly outperforms SURE.

Answer to RQ2: Overall, the experimental results show that: (1) GREClue outperforms all failure indexing methods on all metrics and datasets; (2) Compared with the state-of-the-art failure indexing method SURE, using the Defects4J-Multifault and SIR datasets, GREClue improves the estimation of fault numbers by 10% to 41%, and GREClue improves FMI , JC , PR , RR , and $F1$ by 11% to 41%, 11% to 41%, 10% to 40%, 10% to 39%, and 10% to 39%, respectively.

7.3 (RQ3) Impact Analysis

To demonstrate the effectiveness of each component, we conducted ablation experiments on the two datasets. We start with the full approach, then remove specific parts separately. Specifically, to observe whether the FSG can help the clustering process, we replace GREClue with GREClue-Suspicion_List, which only inputs the sequence features of suspiciousness ranking lists into the entropy-based deep clustering model. Next, we replace GREClue with GREClue-K-medoids, which first predicts the number of clusters using the data point density estimation method and then performs clustering using K -medoids. The alternative clustering method is widely used by recent baselines [22, 54, 55]. Finally, we replace GREClue with GREClue-RMALL, which removes FSG and entropy-based deep clustering method, and uses K -medoids for clustering like GREClue-K-medoids. The results are shown in Table 6, the upper half of the table is the result for Codebert-based GREClue and the lower half of the table is the result for Starcoder-based GREClue.

From the table, we can make the following observations. First, the performance of GREClue decreases a lot after deleting the entire FSG. In terms of predicting the number of faults, the GREClue makes the correct prediction for 6 to 12 fewer faulty versions. In terms of clustering results, the performance of GREClue in FMI , JC , PR , RR , and $F1$ all decreases by 8% to 17%. This shows that FSG can effectively capture semantic information and runtime information of failures, which is crucial for helping GREClue divide the failures based on the root causes. Second, the performance of GREClue decreases significantly after replacing the entropy-based deep clustering method with K -medoids. In terms of predicting the number of faults, the GREClue makes the correct prediction for 14 to 21 fewer faulty versions. In terms of clustering results, the performance of GREClue in

FMI , JC , and $F1$ all decreases by 11% to 30%. This shows that entropy-based deep clustering methods are more suitable for failure indexing tasks compared to traditional clustering methods. Finally, after removing all core components, the performance of GREClue drops most significantly.

Answer to RQ3: The performance of GREClue after removing different components shows that: all major components of the proposed method contribute positively to the final results.

7.4 (RQ4) Parallel Debugging Effectiveness

To show that failure indexing approaches can effectively assist in parallel debugging, we simulate the parallel debugging process based on the results of the failure indexing approaches. We refer to previous parallel debugging works [28, 30] and specify the following parallel debugging process: a) cluster the failing tests according to the results of the failure indexing approaches; b) each cluster of failed tests and all passed tests form the corresponding test suite; c) generate a corresponding suspiciousness ranking list for each test suite; d) fix the faulty code for each suspiciousness ranking list in parallel and calculate the parallel debugging cost. Due to the imperfect clustering, the above steps are performed in multiple iterations until all faults are fixed. We use the FF score proposed by Jones et al. [30] to measure the relative cost savings of parallel debugging. Specifically, FF is computed as the sum of the maximum developer expense at each debugging iteration:

$$Expense_f = \frac{rank_f}{size(program)} \quad (22)$$

$$FF = \sum_{i=1}^{iterations} \max \left\{ Expense_f \mid f \text{ is a targeted fault at iteration } i \right\} \quad (23)$$

where $rank_f$ is the ranking of the fault f in the suspiciousness ranking list and $size(program)$ is the number of all code lines. When a fault involves multiple instructions, following the widely adopted method by previous works [15, 30, 50], we calculate the $Expense$ as the first faulty instruction found in the instructions sorted by decreasing suspiciousness. We sampled 30 multi-fault projects from the Defects4J-Multifault dataset, including the same number (i.e., 6) for each of the 5 types of multi-fault projects (i.e., faulty programs with the number of faults ranging from 1 to 5). To compare, we reproduce the two best baselines Reclues and SURE, and compare them with GREClue in parallel debugging. We calculate the mean (FF_{mean}) and standard deviation (FF_{sta}) of the FF score for each failure indexing approach, and the results are shown in Table 7.

Table 7. FF value of the failure indexing approaches.

Model	FF_{mean}	FF_{sta}
Reclues	24.64	19.25
SURE	23.35	17.43
GREClue-Codebert(our)	17.67	13.48
GREClue-Starcode3B(our)	15.36	11.38

to Reclues and SURE. The results show a significant reduction in parallel debugging costs based on the GREClue results. In other words, GREClue is more effective in assisting in parallel debugging.

Answer to RQ4: The experimental results demonstrate that GREClue incurs the lowest cost in parallel debugging, suggesting that GREClue can more effectively assist in parallel debugging.

8 Discussion

8.1 Overhead and Scalability of GREClue

This section analyzes the cost and scalability of GREClue to show its practicality. The cost of GREClue consists of three aspects: time cost, storage cost, and hardware requirements. The time

cost of GREClue mainly consists of three parts: FSG construction, feature extraction, and clustering. On our machine (Intel Xeon Gold 6226R CPU and NVIDIA RTX A6000 GPU), collecting coverage and computing the suspiciousness list takes 5 minutes per failing test on average, and FSG construction adds 0.9 seconds. Feature extraction takes 2.8 and 0.8 seconds per failing test for GREClue–Starcoder3B and GREClue–Codebert respectively, while clustering adds only 0.1 seconds per instance at inference. Compared with the average inference time of 294.52 seconds achieved by SURE, GREClue has a slightly higher average inference time of 303.8 seconds, but still remains practical and provides substantial performance improvements. In terms of storage, an FSG contains 463 nodes on average and occupies about 371KB. Considering training artifacts (e.g., checkpoints and cached data), the full pipeline requires approximately 60GB of storage. For hardware, training GREClue–Starcoder3B uses two RTX A6000 GPUs (48 GB each) and training GREClue–Codebert uses one, and both variants run inference on a single RTX A6000. In terms of scalability, the cost of GREClue depends on the program size and the number of tests. While our datasets are project-level (tens of thousands of lines), class- or function-level debugging typically involves far less code, so GREClue processes fewer suspicious lines for FSG construction and representation extraction, reducing time and storage overhead. Fewer tests also mean fewer failing tests and thus fewer FSGs. For example, Closure–2–3 has 4 failed tests versus 7 in Closure–2–3–4, reducing GREClue–Starcoder3B inference time by 10.4 seconds and FSG storage by 937KB.

8.2 Limitations

While GREClue achieves SOTA performance, several limitations should be acknowledged. First, the entropy-based clustering is not fully parameter-free: the merging/splitting thresholds λ_1 and λ_2 are selected via grid search on training projects. Empirically, GREClue is stable for $\lambda_1 \in [2, 5]$ and $\lambda_2 \in [3, 5]$, whereas settings outside this range (e.g., $\lambda_1 \leq 1$ or $\lambda_2 \geq 6$) can noticeably degrade performance. This implies that retuning may be necessary on datasets with substantially different failure distributions. Second, the entropy-based clustering relies on posterior probabilities as pseudo-labels, which may amplify early misassignments when the initial clustering is inaccurate. We mitigate this risk by using highly discriminative failure features (FSG plus language-model embeddings) and adopting soft posteriors that allow gradual correction during training. Third, to model indirect dependencies, we use lightweight type-based approximations (e.g., RTA-style call resolution and type matching). These approximations can be imprecise in complex scenarios as pointed out by [37], thus introducing spurious call- and data-flow edges in the FSG.

8.3 Threats to Validity

External Validity. The major threat to external validity is related with the generalizability. To minimize this threat, 1) we conduct experiments using two widely used datasets (i.e., Defects4J-Multifault and SIR datasets) for evaluating failure indexing techniques, and they contain Java and C code respectively; 2) we consider two code language models (i.e., Codebert and Starcoder3B); 3) we also select eight failure indexing methods as the baselines. Due to the limited availability of mature failure-indexing benchmarks, our evaluation currently focuses on Java and C code. Nevertheless, the proposed FSG representation and entropy-based clustering are language-agnostic in principle. We will validate GREClue on other major programming languages in the future.

Internal Validity. The key threat to internal validity concerns the use of code language models, and the issue is that the code language models may possibly know the content of the used datasets in advance. Although this is a common potential issue faced by most studies using LLMs, our study is not much affected by it. Note that we do not change the code language model in the ablation study, and the results of the ablation study clearly show the usefulness of the core components of GREClue, suggesting that our approach itself provides new insights into the failure indexing task.

Construct Validity. One threat to construct validity concerns to the adopted evaluation metrics. To reduce the threat, we use multiple metrics (*i.e.*, FMI, JC, PR, and RR) to measure clustering effectiveness, and these metrics have been extensively used in previous works. Another threat to construct validity is related with the randomness when evaluating GREClue. To cope with this threat, following the guideline in [9], we randomly split the SIR dataset 30 times and report the average result over these splits. Besides, the statistical analysis has also been conducted.

9 Related Work

Failure Representation. The first CC-based failure representation method was proposed by Podgurski et al. [46], and since then has been continuously explored by the community. For instance, Liu et al. [36] use a digital vector containing 0 and 1 to describe the coverage of the failed test. Later, Liu and Han introduce the SD-based failure representation method which considers two failures as similar if they suggest roughly the same fault location [35]. SD-based methods have also been widely explored by researchers [22, 30, 31, 69], and the focus is how to more accurately suggest the fault location. More recently, Song et al. [54, 55] propose using program memory information to represent failures. All of these failure representation methods do not capture the deep failure semantic well, and this paper designs the failure semantic graph to overcome the issue. Meanwhile, recent structure-aware code representation methods, such as Code2Vec [4], also learn code embeddings from syntactic or data-flow structures. However, these methods focus on understanding code snippets and do not incorporate failure-specific information, making them unsuitable for failure indexing tasks. Note that there exist some recent works that introduce additional information to facilitate failure indexing, including code-independent features in regression testing [25] and historical features in continuous integration [5]. However, the required additional information cannot always be available and thus are out of our research scope.

Failure Clustering Algorithm. Many research efforts have been devoted to failure clustering algorithms. Liu et al. [35] propose a new algorithm based on R -proximity evaluation. Nicholas et al. [17] propose that the algorithm requires a pre-clustering step to eliminate failures that cause cluster impurities. Gao et al. [22] propose a new K -medoids algorithm, which sets cluster centers by calculating the potential values of data points and divides the data within the cluster at the same time. Song et al. [54, 55] predict the number of faults based on density and use the K -Medoids algorithm to partition failures. However, even the most advanced algorithms mainly cluster by presetting cluster centers [22, 55], which are difficult to adapt to diverse failure clustering scenarios in practice. To address the issue, this paper introduces an entropy-based deep clustering procedure.

10 Conclusion

This paper proposes GREClue, a novel failure indexing approach with graph-based failure representation and entropy-based deep clustering. GREClue overcomes the issues of existing failure indexing techniques in order. For failure representation, GREClue designs failure semantic graph, a new graph representation that effectively contains semantic information and runtime information of failures. Based on the FSG, GREClue further consists of an entropy based deep clustering component, which can accurately cluster failed tests without presetting cluster centers. An extensive evaluation shows that GREClue achieves state-of-the-art performance in failure indexing tasks.

11 Data Availability

The replication package for our work is available at <https://github.com/HumblePLSE/GREClue>.

Acknowledgments

We greatly appreciate the anonymous reviewers for their insightful comments. This work was supported by National Natural Science Foundation of China (Grants No. 62102233 and No. U24A20244).

References

- [1] Mohiuddin Ahmed, Raihan Seraj, and Syed Mohammed Shamsul Islam. 2020. The k-means algorithm: A comprehensive survey and performance evaluation. *Electronics* 9, 8 (2020), 1295. doi:10.3390/electronics9081295
- [2] Miltiadis Allamanis, Earl T. Barr, Christian Bird, and Charles Sutton. 2014. Learning Natural Coding Conventions. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. Association for Computing Machinery, 281–293. doi:10.1145/2635868.2635883
- [3] Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, and Charles Sutton. 2018. A Survey of Machine Learning for Big Code and Naturalness. *ACM Comput. Surv.* (2018). doi:10.1145/3212695
- [4] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2019. Code2vec: Learning Distributed Representations of Code. *Proc. ACM Program. Lang.* POPL (2019), 29 pages. doi:10.1145/3290353
- [5] Gabin An, Juyeon Yoon, Jeongju Sohn, Jingun Hong, Dongwon Hwang, and Shin Yoo. 2022. Automatically identifying shared root causes of test breakages in SAP HANA. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice* (Pittsburgh, Pennsylvania) (ICSE-SEIP '22). Association for Computing Machinery, New York, NY, USA, 65–74. doi:10.1145/3510457.3513051
- [6] Gabin An, Juyeon Yoon, and Shin Yoo. 2021. Searching for multi-fault programs in defects4j. In *International Symposium on Search Based Software Engineering*. Springer, 153–158. doi:10.1007/978-3-030-88106-1_11
- [7] J.H. Andrews, L.C. Briand, Y. Labiche, and A.S. Namin. 2006. Using Mutation Analysis for Assessing and Comparing Testing Coverage Criteria. *IEEE Transactions on Software Engineering* 32, 8 (2006), 608–624. doi:10.1109/TSE.2006.83
- [8] J. H. Andrews, L. C. Briand, and Y. Labiche. 2005. Is mutation an appropriate tool for testing experiments?. In *Proceedings of the 27th International Conference on Software Engineering* (St. Louis, MO, USA) (ICSE '05). Association for Computing Machinery, New York, NY, USA, 402–411. doi:10.1145/1062455.1062530
- [9] Andrea Arcuri and Lionel Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd International Conference on Software Engineering* (Waikiki, Honolulu, HI, USA) (ICSE '11). Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/1985793.1985795
- [10] R Babuka, PJ Van der Veen, and Uzey Kaymak. 2002. Improved covariance estimation for Gustafson-Kessel clustering. In *2002 IEEE World Congress on Computational Intelligence. 2002 IEEE International Conference on Fuzzy Systems. FUZZ-IEEE'02. Proceedings* (Cat. No. 02CH37291), Vol. 2. IEEE, 1081–1085. doi:10.1109/FUZZ.2002.1006654
- [11] Tal Ben-Nun, Alice Shoshana Jakobovits, and Torsten Hoefler. 2018. Neural Code Comprehension: A Learnable Representation of Code Semantics. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) (NIPS'18). Curran Associates Inc., 3589–3601.
- [12] Benjamin Bichsel, Veselin Raychev, Petar Tsankov, and Martin Vechev. 2016. Statistical Deobfuscation of Android Applications. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (Vienna, Austria) (CCS '16). Association for Computing Machinery, 343–355. doi:10.1145/2976749.2978422
- [13] Kwonsoo Chae, Hakjoo Oh, Kihong Heo, and Hongseok Yang. 2017. Automatically Generating Features for Learning Program Analysis Heuristics for C-like Languages. *Proc. ACM Program. Lang.* OOPSLA (2017), 25 pages. doi:10.1145/3133925
- [14] Yaniv David, Uri Alon, and Eran Yahav. 2020. Neural reverse engineering of stripped binaries using augmented control flow graphs. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–28. doi:10.1145/3428293
- [15] Nicholas DiGiuseppe and James A. Jones. 2011. On the influence of multiple faults on coverage-based fault localization. In *Proceedings of the 2011 International Symposium on Software Testing and Analysis* (Toronto, Ontario, Canada) (ISSTA '11). Association for Computing Machinery, New York, NY, USA, 210–220. doi:10.1145/2001420.2001446
- [16] Nicholas DiGiuseppe and James A. Jones. 2012. Concept-based failure clustering. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering* (Cary, North Carolina) (FSE '12). Association for Computing Machinery, New York, NY, USA, Article 29, 4 pages. doi:10.1145/2393596.2393629
- [17] Nicholas DiGiuseppe and James A Jones. 2012. Software behavior and failure clustering: An empirical study of fault causality. In *2012 IEEE Fifth International Conference on Software Testing, Verification and Validation*. IEEE, 191–200. doi:10.1109/ICST.2012.99
- [18] Amer Diwan, Kathryn S. McKinley, and J. Eliot B. Moss. 1998. Type-based alias analysis. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation* (Montreal, Quebec, Canada) (PLDI '98). Association for Computing Machinery, New York, NY, USA, 106–117. doi:10.1145/277650.277670
- [19] Hyunsook Do, Sebastian Elbaum, and Gregg Rothermel. 2005. Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact. *Empirical Software Engineering* 10, 4 (2005), 405–435.
- [20] Yali Du and Zhongxing Yu. 2023. Pre-training Code Representation with Semantic Flow Graph for Effective Bug Localization. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) (ESEC/FSE 2023). Association for Computing Machinery, New York, NY, USA, 579–591. doi:10.1145/3611643.3616338

- [21] Yang Feng, James Jones, Zhenyu Chen, and Chunrong Fang. 2018. An Empirical Study on Software Failure Classification with Multi-label and Problem-Transformation Techniques. In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*. 320–330. doi:10.1109/ICST.2018.00039
- [22] Ruizhi Gao and W Eric Wong. 2018. MSeer: an advanced technique for locating multiple bugs in parallel. In *Proceedings of the 40th International Conference on Software Engineering*. 1064–1064. doi:10.1145/3180155.3182552
- [23] GBLLM. 2025. Tree-sitter. <https://github.com/tree-sitter/tree-sitter>.
- [24] Joydeep Ghosh and Alexander Liu. 2009. K-means. In *The top ten algorithms in data mining*. Chapman and Hall/CRC, 35–50.
- [25] Mojdeh Golagha, Constantin Lehnhoff, Alexander Pretschner, and Hermann Ilmberger. 2019. Failure clustering without coverage. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (Beijing, China) (ISSTA 2019)*. Association for Computing Machinery, New York, NY, USA, 134–145. doi:10.1145/3293882.3330561
- [26] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).
- [27] Jin Guo, Jinghui Cheng, and Jane Cleland-Huang. 2017. Semantically Enhanced Software Traceability Using Deep Learning Techniques. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. 3–14. doi:10.1109/ICSE.2017.9
- [28] Wolfgang Högerle, Friedrich Steimann, and Marcus Frenkel. 2014. More Debugging in Parallel. In *2014 IEEE 25th International Symposium on Software Reliability Engineering*. 133–143. doi:10.1109/ISSRE.2014.29
- [29] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1430–1442. doi:10.1109/ICSE48619.2023.00125
- [30] James A. Jones, James F. Bowring, and Mary Jean Harrold. 2007. Debugging in Parallel. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis (London, United Kingdom) (ISSTA '07)*. Association for Computing Machinery, New York, NY, USA, 16–26. doi:10.1145/1273463.1273468
- [31] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering (Long Beach, CA, USA) (ASE '05)*. Association for Computing Machinery, New York, NY, USA, 273–282. doi:10.1145/1101908.1101949
- [32] René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*. 437–440. doi:10.1145/2610384.2628055
- [33] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [34] Si-Mohamed Lamraoui and Shin Nakajima. 2016. A formula-based approach for automatic fault localization of multi-fault programs. *Journal of Information Processing* 24, 1 (2016), 88–98. doi:10.2197/ipsjip.24.88
- [35] Chao Liu and Jiawei Han. 2006. Failure proximity: a fault localization-based approach. In *Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering*. 46–56. doi:10.1145/1181775.1181782
- [36] Chao Liu, Xiangyu Zhang, and Jiawei Han. 2008. A systematic study of failure proximity. *IEEE Transactions on Software Engineering* 34, 6 (2008), 826–843. doi:10.1109/TSE.2008.66
- [37] Dinghao Liu, Shouling Ji, Kangjie Lu, and Qinming He. 2024. Improving Indirect-Call Analysis in LLVM with Type and Data-Flow Co-Analysis. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 5895–5912. <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-dinghao-improving>
- [38] Wei Liu, Ailun Yu, Daoguang Zan, Bo Shen, Wei Zhang, Haiyan Zhao, Zhi Jin, and Qianxiang Wang. 2024. Graphcoder: Enhancing repository-level code completion via code context graph-based retrieval and language model. *arXiv preprint arXiv:2406.07003* (2024).
- [39] Wei Ma, Mengjie Zhao, Ezekiel Soremekun, Qiang Hu, Jie M Zhang, Mike Papadakis, Maxime Cordy, Xiaofei Xie, and Yves Le Traon. 2022. Graphcode2vec: Generic code embedding via lexical and program dependence analyses. In *Proceedings of the 19th international conference on mining software repositories*. 524–536. doi:10.1145/3524842.3528456
- [40] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, Nov (2008), 2579–2605. doi:arXiv:2108.01301
- [41] Xiaoguang Mao, Yan Lei, Ziyang Dai, Yuhua Qi, and Chengsong Wang. 2014. Slice-based statistical fault localization. *Journal of Systems and Software* 89 (2014), 51–62.
- [42] Arvind Neelakantan, Tao Xu, Raul Puri, Alec Radford, Jesse Michael Han, Jerry Tworek, Qiming Yuan, Nikolas Tezak, Jong Wook Kim, Chris Hallacy, et al. 2022. Text and code embeddings by contrastive pre-training. *arXiv preprint arXiv:2201.10005* (2022).
- [43] Mike Papadakis, Marinos Kintis, Jie Zhang, Yue Jia, Yves Le Traon, and Mark Harman. 2019. Chapter Six - Mutation Testing Advances: An Analysis and Survey. *Advances in Computers*, Vol. 112. Elsevier, 275–378. doi:10.1016/bs.adcom.

2018.03.015

- [44] Renaud Pawlak, Martin Monperrus, Nicolas Petitprez, Carlos Noguera, and Lionel Seinturier. 2015. Spoon: A Library for Implementing Analyses and Transformations of Java Source Code. *Software: Practice and Experience* 46 (2015), 1155–1179. doi:10.1002/spe.2346
- [45] Hanyu Pei, Beibei Yin, Min Xie, and Kai-Yuan Cai. 2021. Dynamic random testing with test case clustering and distance-based parameter adjustment. *Information and Software Technology* 131 (2021), 106470. doi:10.1016/j.infsof.2020.106470
- [46] Andy Podgurski, David Leon, Patrick Francis, Wes Masri, Melinda Minch, Jiayang Sun, and Bin Wang. 2003. Automated support for classifying software failure reports. In *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE, 465–475.
- [47] Michael Pradel and Koushik Sen. 2018. DeepBugs: a learning approach to name-based bug detection. *Proc. ACM Program. Lang.* 2, OOPSLA, Article 147 (Oct. 2018), 25 pages. doi:10.1145/3276517
- [48] Yannan Pu, Jian Sun, Niansheng Tang, and Zongben Xu. 2023. Deep expectation-maximization network for unsupervised image segmentation and clustering. *Image and Vision Computing* 135 (2023), 104717. doi:10.1016/j.imavis.2023.104717
- [49] PyTorch. 2023. . <https://pytorch.org/>.
- [50] Raul Santelices, James A. Jones, Yanbing Yu, and Mary Jean Harrold. 2009. Lightweight fault-localization using multiple coverage types. In *2009 IEEE 31st International Conference on Software Engineering*. 56–66. doi:10.1109/ICSE.2009.5070508
- [51] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE transactions on neural networks* 20, 1 (2008), 61–80. doi:10.1109/TNN.2008.2005605
- [52] Ting Shu, Tiantian Ye, Zuohua Ding, and Jinsong Xia. 2016. Fault localization based on statement frequency. *Inf. Sci.* 360, C (Sept. 2016), 43–56. doi:10.1016/j.ins.2016.04.023
- [53] Yi Song, Xiaoyuan Xie, Xihao Zhang, Quanming Liu, and Ruizhi Gao. 2023. Evolving Ranking-Based Failure Proximities for Better Clustering in Fault Isolation. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (Rochester, MI, USA) (ASE '22). Association for Computing Machinery, New York, NY, USA, Article 41, 13 pages. doi:10.1145/3551349.3556922
- [54] Yi Song, Xihao Zhang, Xiaoyuan Xie, Songqiang Chen, Quanming Liu, and Ruizhi Gao. 2024. SURE: A Visualized Failure Indexing Approach Using Program Memory Spectrum. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–43. doi:10.1145/3676958
- [55] Yi Song, Xihao Zhang, Xiaoyuan Xie, Quanming Liu, Ruizhi Gao, and Chenliang Xing. 2024. ReClues: Representing and indexing failures in parallel debugging with program variables. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (ICSE '24). Association for Computing Machinery, New York, NY, USA, Article 111, 13 pages. doi:10.1145/3597503.3639098
- [56] Friedrich Steimann and Marcus Frenkel. 2012. Improving Coverage-Based Localization of Multiple Faults Using Algorithms from Integer Linear Programming. In *2012 IEEE 23rd International Symposium on Software Reliability Engineering*. 121–130. doi:10.1109/ISSRE.2012.28
- [57] Vijay Sundaresan, Laurie Hendren, Chrislain Razafimahefa, Raja Vallée-Rai, Patrick Lam, Etienne Gagnon, and Charles Godin. 2000. Practical virtual method call resolution for Java. *ACM SIGPLAN Notices* 35, 10 (2000), 264–280. doi:10.1145/354222.353189
- [58] Sindhu Tipirneni, Ming Zhu, and Chandan K Reddy. 2024. Structcoder: Structure-aware transformer for code generation. *ACM Transactions on Knowledge Discovery from Data* 18, 3 (2024), 1–20. doi:10.1145/3636430
- [59] Dumindu Tissera, Kasun Vithanage, Rukshan Wijesinghe, Alex Xavier, Sanath Jayasena, Subha Fernando, and Ranga Rodrigo. 2022. Neural mixture models with expectation-maximization for end-to-end deep clustering. *Neurocomputing* 505 (2022), 249–262. doi:10.1016/j.neucom.2022.07.017
- [60] Simon Urli, Zhongxing Yu, Lionel Seinturier, and Martin Monperrus. 2018. How to design a program repair bot? insights from the repairnator project. In *2018 IEEE/ACM 40th international conference on software engineering: software engineering in practice Track (ICSE-SEIP)*. IEEE, 95–104. doi:10.1145/3183519.3183540
- [61] Zekeriya Uykun. 2021. Fusion of centroid-based clustering with graph clustering: An expectation-maximization-based hybrid clustering. *IEEE Transactions on Neural Networks and Learning Systems* 34, 8 (2021), 4068–4082. doi:10.1109/TNNLS.2021.3121224
- [62] András Vargha and Harold D Delaney. 2000. A critique and improvement of the CL common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics* 25, 2 (2000), 101–132. doi:10.3102/10769986025002101
- [63] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [64] Iris Vessey. 1985. Expertise in debugging computer programs: A process analysis. *International Journal of Man-Machine Studies* 23, 5 (1985), 459–494. doi:10.1016/S0020-7373(85)80054-7

- [65] Jeffrey M. Voas. 1992. PIE: A Dynamic Failure-Based Technique. *IEEE Trans. Softw. Eng.* 18, 8 (Aug. 1992), 717–727. doi:10.1109/32.153381
- [66] Wanzhi Wen. 2012. Software fault localization based on program slicing spectrum. In *2012 34th International Conference on Software Engineering (ICSE)*. 1511–1514. doi:10.1109/ICSE.2012.6227049
- [67] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics bulletin* 1, 6 (1945), 80–83.
- [68] W. Eric Wong, Vidroha Debroy, Ruizhi Gao, and Yihao Li. 2014. The DStar Method for Effective Software Fault Localization. *IEEE Transactions on Reliability* 63, 1 (2014), 290–308. doi:10.1109/TR.2013.2285319
- [69] W. Eric Wong, Vidroha Debroy, and Dianxiang Xu. 2012. Towards Better Fault Localization: A Crosstab-Based Statistical Approach. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 42, 3 (2012), 378–396. doi:10.1109/TSMCC.2011.2118751
- [70] Hui Xiong, Junjie Wu, and Jian Chen. 2006. K-means clustering versus validation measures: a data distribution perspective. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*. 779–784. doi:10.1145/1150402.1150503
- [71] Zhenyu Yang, Jingwen Wu, Zhen Yang, and Zhongxing Yu. 2026. Parameter-Efficient Fine-Tuning with Attributed Patch Semantic Graph for Automated Patch Correctness Assessment. *IEEE Transactions on Software Engineering* (2026). doi:10.1109/TSE.2026.3684016
- [72] Zhongxing Yu, Chenggang Bai, and Kai-Yuan Cai. 2013. Mutation-oriented test data augmentation for GUI software fault localization. *Information and Software Technology* 55, 12 (2013), 2076–2098. doi:10.1016/j.infsof.2013.07.004
- [73] Zhongxing Yu, Chenggang Bai, and Kai-Yuan Cai. 2015. Does the Failing Test Execute a Single or Multiple Faults? An Approach to Classifying Failing Tests. In *Proceedings of the 37th International Conference on Software Engineering - Volume 1 (Florence, Italy) (ICSE '15)*. IEEE Press, 924–935. doi:10.1109/ICSE.2015.102
- [74] Zhongxing Yu, Hai Hu, Chenggang Bai, Kai-Yuan Cai, and W. Eric Wong. 2011. GUI Software Fault Localization Using N-gram Analysis. In *2011 IEEE 13th International Symposium on High-Assurance Systems Engineering*. 325–332. doi:10.1109/HASE.2011.29
- [75] Zhongxing Yu, Matias Martinez, Zimin Chen, Tegawendé F. Bissyandé, and Martin Monperrus. 2023. Learning the Relation Between Code Features and Code Transforms With Structured Prediction. *IEEE Transactions on Software Engineering* 49, 7 (2023), 3872–3900. doi:10.1109/TSE.2023.3275380
- [76] Zhongxing Yu, Matias Martinez, Benjamin Danglot, Thomas Durieux, and Martin Monperrus. 2019. Alleviating Patch Overfitting with Automatic Test Generation: A Study of Feasibility and Effectiveness for the Nopol Repair System. *Empirical Softw. Engg.* 24, 1 (feb 2019), 33–67. doi:10.1007/s10664-018-9619-4
- [77] Yizhuo Zhai, Yu Hao, Hang Zhang, Daimeng Wang, Chengyu Song, Zhiyun Qian, Mohsen Lesani, Srikanth V Krishnamurthy, and Paul Yu. 2020. Ubitect: a precise and scalable method to detect use-before-initialization bugs in linux kernel. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 221–232. doi:10.1145/3368089.3409686
- [78] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B Tenenbaum, and Chuang Gan. 2023. Planning with large language models for code generation. *arXiv preprint arXiv:2303.05510* (2023).

Received 2025-09-12; accepted 2026-03-24